Design of ALU Components. Multiplication and Division

1 Purpose

The purpose of this laboratory work is to show how the multiplication and division operations are implemented in the Arithmetic Logic Unit (ALU). For each operation, the algorithm is first described and illustrated as a flowchart, and then the hardware block design that implements it is presented. The designs are then implemented and simulated in the exercises at the end of this work.

2 Multiplication

The multiplication of binary numbers is similar to that of decimal numbers, but simpler: since each digit of the multiplier is either 0 or 1, every partial product is either 0 or a shifted copy of the multiplicand. The product of two n-bit unsigned numbers has at most $2n$ bits. There are various methods of multiplying two n-bit numbers:

- Shift-and-Add Multiplication
- Booth's Technique
- Higher-Radix Multiplication
- Array Multiplier
- Wallace Tree

In the following subsections, two of these methods are explained in detail: Shift-and-Add Multiplication, the simplest sequential method, and the Wallace Tree, a fast parallel method.

2.1 Shift-and-Add Multiplication

Shift-and-Add Multiplication is one of the most basic and simplest methods for multiplying two numbers. It follows the pencil-and-paper method: the multiplicand X is multiplied by each digit of the multiplier Y in turn, each intermediate product is placed one position to the left of the previous one, and finally all the intermediate products are added together to get the result. In binary, multiplying X by a single bit is trivial: the intermediate product is X if the bit is 1 and 0 otherwise. Therefore, the whole multiplication reduces to a sequence of additions and shifts.

For two n-bit operands, the algorithm uses the following registers:

- B ($2n$ bits) holds the multiplicand X; it is shifted left after each step, so that it is always aligned with the weight of the multiplier bit currently examined;
- Q (n bits) holds the multiplier Y; it is shifted right after each step, so that the bit to be examined is always the least significant one, Q_0;
- A ($2n$ bits) accumulates the intermediate products and holds the final product;
- N counts the remaining steps and is initialized with n.

In each of the n iterations, the bit Q_0 is checked. If $Q_0 = 1$, the multiplicand is added to the partial product ($A \leftarrow A + B$); otherwise, A remains unchanged. Then B is shifted left, Q is shifted right and the counter N is decremented. The algorithm stops when N reaches 0, and the product is found in A. The steps of the Shift-and-Add Multiplication algorithm are shown in Figure 1.

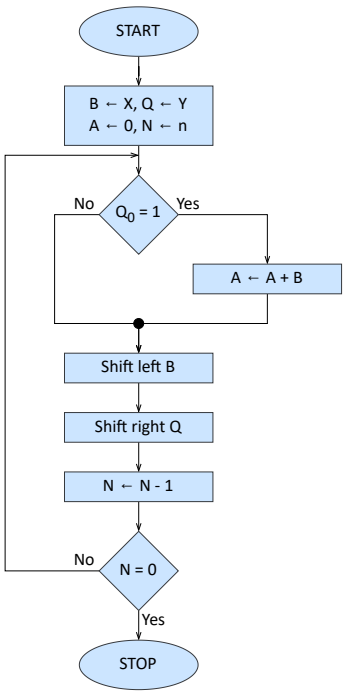


Figure 1: Shift-and-Add Multiplication Algorithm

As an example, Table 1 shows the content of the registers after each iteration when multiplying $X = 1101_2$ (13) by $Y = 1011_2$ (11), for $n = 4$. The final content of A is $10001111_2 = 143 = 13 \cdot 11$.

Table 1: Shift-and-Add Multiplication of 1101_2 by 1011_2

Iteration	Q_0	A	B	Q
initial	–	00000000	00001101	1011
1	1	00001101	00011010	0101
2	1	00100111	00110100	0010
3	0	00100111	01101000	0001
4	1	10001111	11010000	0000

The block design of the shift-and-add multiplier is illustrated in Figure 2. It consists of the three registers described above, a $2n$-bit ALU and a Control Unit. The ALU adds the contents of A and B, and the result is written back into A only when the Control Unit activates the *Load* signal, i.e. when $Q_0 = 1$. The Control Unit examines Q_0, generates the *Add*, *Load*, *Shift left* (for B) and *Shift right* (for Q) control signals, and counts the iterations. Thus, the multiplication of two n-bit numbers takes n iterations, each consisting of an optional addition followed by shifts.

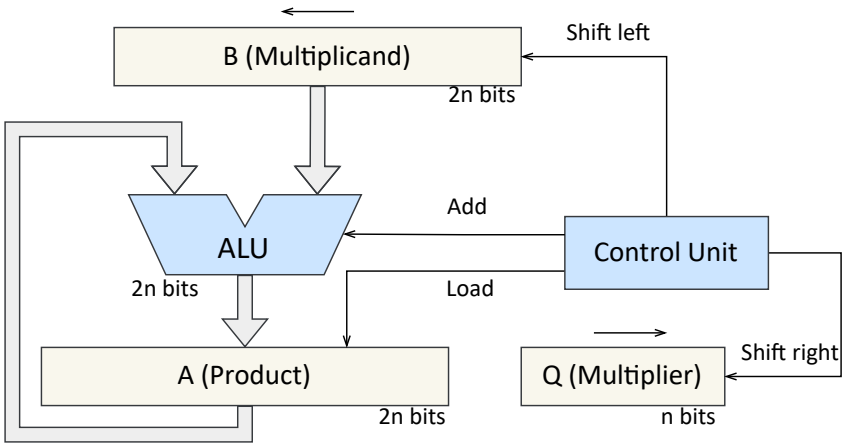


Figure 2: Block design of the shift-and-add multiplication technique

2.2 Wallace Tree Multiplication

The main drawback of the Shift-and-Add method is that the intermediate products are added sequentially, one per iteration, and each addition has to propagate the carry across the whole width of the adder. The **Wallace Tree** technique speeds up the multiplication by generating all the partial products in parallel (bit j of the k-th partial product is simply $x_j y_k$, obtained with an AND gate) and adding them with the help of multiple levels of Carry Save Adders (CSAs). A Carry Save Adder is a row of independent full adders that takes three numbers and reduces them to two numbers, a sum word S and a carry word C, without propagating the carries; therefore, its delay is that of a single full adder, regardless of the width of the operands. The Wallace Tree works as follows:

- at each level of the tree, the numbers are grouped into three and each group is reduced to two numbers by a CSA
- the levels continue until only 2 numbers are left to be added
- a Carry Propagate Adder (CPA) is used to add the last 2 numbers and deliver the final result

An example for multiplying two 4-bit numbers using the Wallace Tree technique is shown in Figure 3, where each partial product bit $x_j y_k$ is fed into a network of full adders (FA). The two upper rows of full adders reduce the partial products of each column to at most two bits, while the bottom row of full adders acts as the final Carry Propagate Adder that delivers the product bits $p_0 \dots p_7$.

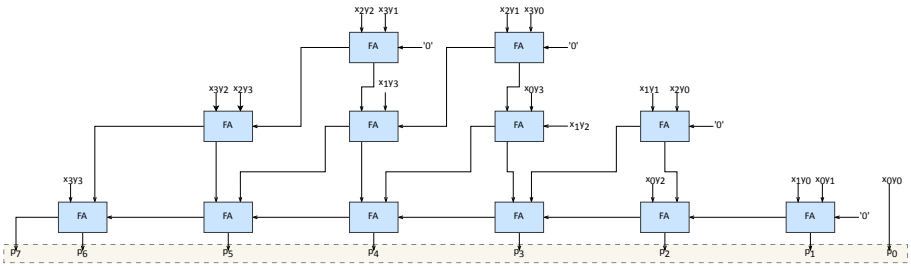


Figure 3: Wallace Tree Multiplier for multiplying two 4-bit numbers

Figure 4 shows the block design of a Wallace Tree for multiplying two 8-bit numbers: the 8 partial products $Xy_0 \dots Xy_7$ are reduced to 6, 4, 3 and finally 2 numbers by four levels of Carry Save Adders, and the last two numbers are added by a Carry Propagate Adder to obtain the product P. Note that the carry word of each CSA is shifted left by one bit before being used, since a carry has double the weight of the sum bit in the same position.

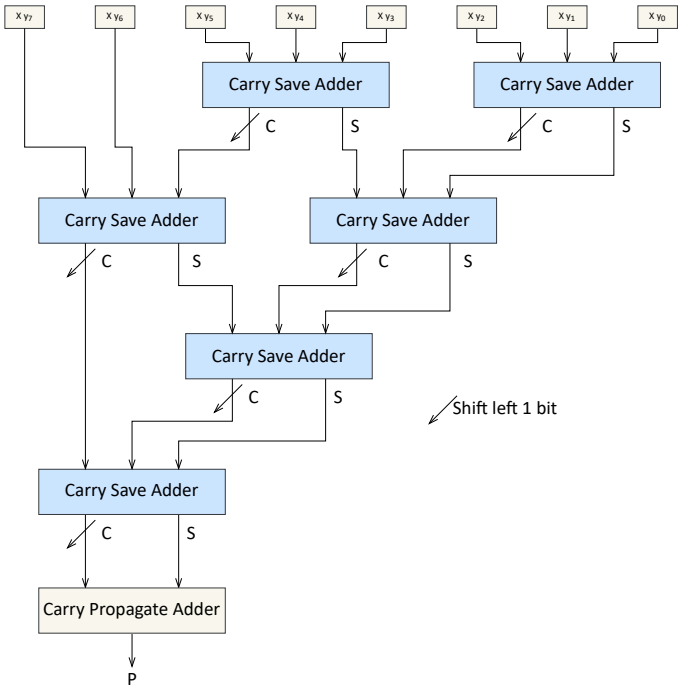


Figure 4: Block diagram of the Wallace Tree design for multiplying two 8-bit numbers

The Wallace Tree method can be combined with other methods to further increase the speed. For example, the Booth technique can be used to produce the partial products, while a Wallace Tree adds the partial products.

3 Division

In any division operation, we have the first operand called *dividend* (X), the second operand called *divisor* (Y) and the results are the *quotient* (Q) and *remainder* (R). The mathematical expression is:

$$X = Q \cdot Y + R, R < Y$$

The algorithm for decimal division is explained below:

1: Choose a digit and subtract the product between this digit and the divisor from the partial remainder
2: If the result is smaller than the divisor, the digit was chosen correctly
3: Otherwise, choose another digit and repeat the subtraction

The binary division algorithm is simpler, because each quotient digit can only be 0 or 1. It is based on repeated subtractions of the divisor Y from the partial remainder R, which are kept only if $Y \leq R$, resulting in a quotient digit of 1 (otherwise the digit is 0). Since the hardware cannot know in advance whether $Y \leq R$, the subtraction is always performed and the sign of the result is checked: if the result is negative, the divisor is added back to *restore* the previous partial remainder. This is why the method is called **Restoring Division**.

For two n-bit operands, the algorithm uses a $2n$-bit register A, initialized with the dividend, which holds the partial remainder; a $2n$-bit register B, initialized with the divisor aligned to its most significant half, which is shifted right after each step; and an n-bit register Q in which the quotient bits are shifted in from the right. Because the divisor starts aligned with the most significant half, $n + 1$ iterations are needed. All the steps of the binary division algorithm are shown in Figure 5.

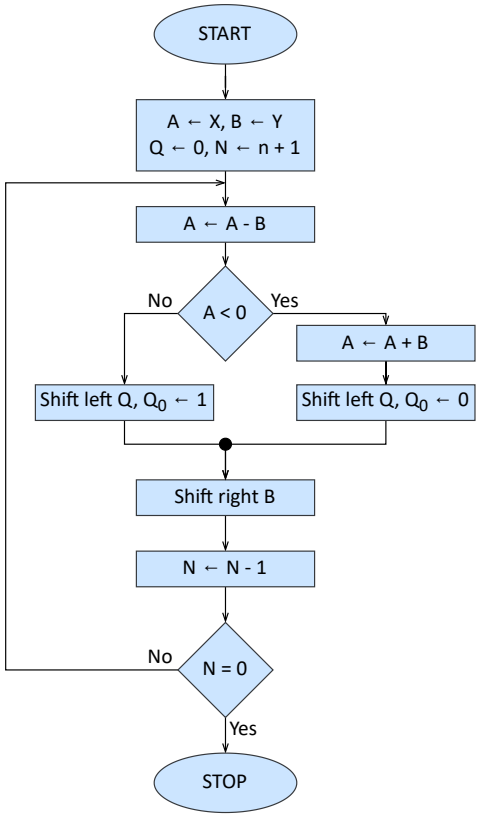


Figure 5: Restoring Division Algorithm

The basic block design of the division operation is illustrated in Figure 6. The $2n$-bit ALU performs either the subtraction $A - B$ or the restoring addition $A + B$, as selected by the Control Unit, and the result is loaded back into A. Based on the sign of the result, the Control Unit shifts the corresponding quotient bit into Q and shifts the divisor register B to the right.

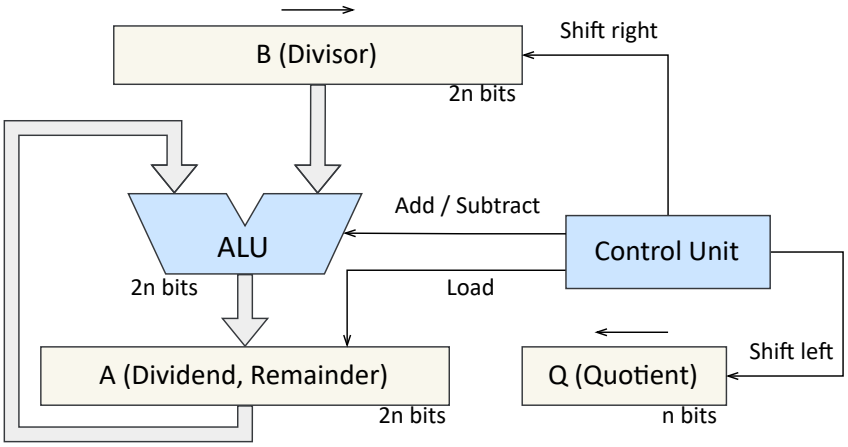


Figure 6: Basic block design of the division operation

Note that this block design can be improved in order to reduce the time and to simplify the hardware needed:

- shifting the partial remainder to the left (instead of shifting the divisor to the right) produces the same alignment and simplifies the hardware of the ALU and of the divisor register (n bits

instead of $2n$);

- since the first step cannot generate a quotient digit of 1, the order of the operations can be switched: first shift, then subtract, which removes one iteration (n iterations instead of $n+1$);
- the size of the A register can be reduced to half and A can be combined with the Q register: the dividend is initially placed in Q, its bits are shifted into A instead of shifting zeros, and both A and Q registers are shifted left together, while the quotient bits fill the positions freed in Q.

The improved version of the algorithm is shown in Figure 7. In each iteration, the register pair AQ is shifted left, the divisor is subtracted from A, and the sign of the result decides the quotient bit Q_0: if $A \geq 0$, then $Q_0 \leftarrow 1$; otherwise, A is restored and $Q_0 \leftarrow 0$. At the end, Q holds the quotient and A holds the remainder.

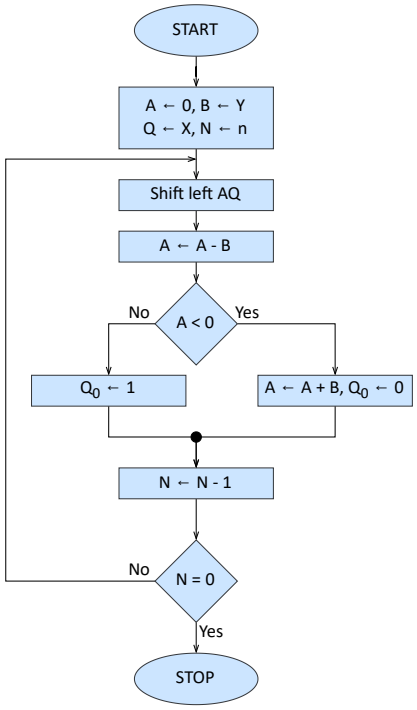


Figure 7: Improved Restoring Division Algorithm

As an example, Table 2 shows the content of the registers after each iteration when dividing $X = 1011_2$ (11) by $Y = 0011_2$ (3), for $n = 4$. The result is the quotient $Q = 0011_2 = 3$ and the remainder $A = 00010_2 = 2$, since $11 = 3 \cdot 3 + 2$.

Table 2: Improved Restoring Division of 1011_2 by 0011_2

Iteration	A after shift	$A - B$	Restore	A	Q
initial	–	–	–	00000	1011
1	00001	< 0	yes	00001	0110
2	00010	< 0	yes	00010	1100
3	00101	≥ 0	no	00010	1001
4	00101	≥ 0	no	00010	0011

Considering the above, the improved block design is shown in Figure 8. Compared to the basic design, the divisor register B has only n bits, the ALU and the remainder register A have $n+1$ bits

(the extra bit holds the sign of the subtraction result), and the Q register holds both the dividend and the quotient, being shifted left together with A.

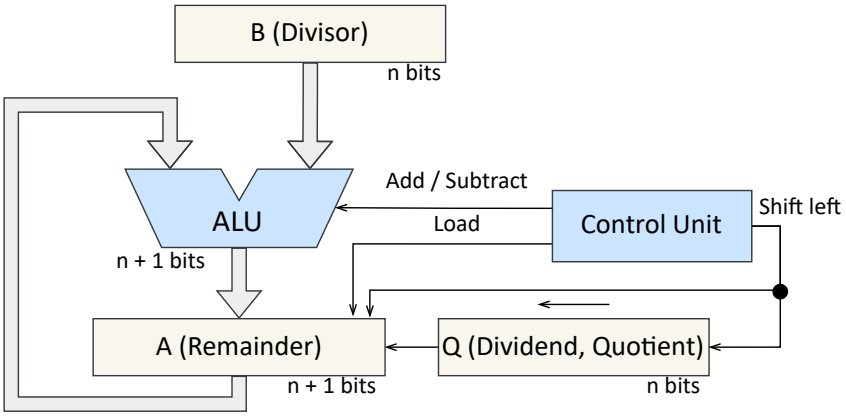


Figure 8: Improved block design of the division operation

4 Exercises

1. **Implement the Shift-and-Add multiplication circuit for 4-bit numbers as depicted in Figure 2. Simulate for pairs of random numbers.**

2. **Implement the Wallace Tree for multiplying two 4-bit numbers depicted in Figure 3. Simulate the design for pairs of random numbers.**

3. **Implement the Division algorithm for 4-bit numbers as depicted in Figure 8. Simulate the design for several pairs of random numbers.**

References

[1] Arithmetic logic unit [accessed Oct. 2024], https://en.wikipedia.org/wiki/Arithmetic_logic_unit

[2] Wallace tree [accessed Oct. 2024], https://en.wikipedia.org/wiki/Wallace_tree

[3] Restoring Division Algorithm For Unsigned Integer [accessed Oct. 2024], https://www.geeksforgeeks.org/restoring-division-algorithm-unsigned-integer/